

---

# Bases de Datos 2

— Transacciones, concurrencia y métodos —  
de control de concurrencia

---

# Repaso ¿Qué es una transacción?

**Transacción:** secuencia de instrucciones (sentencias SQL) relacionadas, que deben ser tratadas como una unidad indivisible.

Para asegurar la integridad de los datos se necesita que el sistema de base de datos mantenga las propiedades ACID de las transacciones: *atomicidad*, *consistencia*, *aislamiento (isolation)*, *durabilidad*.

Estas propiedades permiten:

- Realizar control de concurrencia.
- Realizar recuperación ante fallos.

# Repaso ACID

- **Atomicidad**: se ejecutan **todas** las operaciones que contiene la transacción o no se ejecuta ninguna.
- **Consistencia**: la ejecución aislada de la transacción (es decir, **sin otra transacción que se ejecute concurrentemente**) **conserva la consistencia de la base de datos**.
- **Aislamiento** (isolation): **Aunque se ejecuten varias transacciones concurrentemente**, el sistema garantiza que para cada par de transacciones  $T_i$  y  $T_j$ , se cumple que para los efectos de  $T_i$ , o bien  $T_j$  ha terminado su ejecución antes de que comience  $T_i$ , o bien que  $T_j$  ha comenzado su ejecución después de que  $T_i$  termine. De este modo, **cada transacción ignora al resto de las transacciones que se ejecuten concurrentemente en el sistema**.
  - $T$  no muestra los cambios hasta que finaliza
  - $T_i$  nunca debe ver las fases intermedias de otra  $T_j$
- **Durabilidad**: **Tras la finalización con éxito de una transacción, los cambios realizados en la base de datos permanecen, incluso si hay fallos en el sistema.**

# Repaso Espacios de datos

Almacenamiento  
secundario (disco)



**Input/Output**

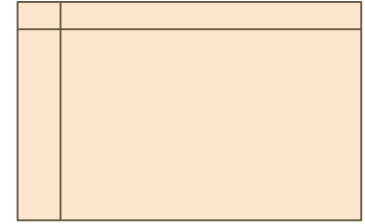


**Buffer de Datos  
Compartido**

**MEMORIA RAM**



**Buffer local transacción  $T_i$**



**read/write**



**Buffer local transacción  $T_j$**



# Ejecuciones concurrentes

Permitir varias transacciones que actualizan concurrentemente los datos provoca complicaciones en la consistencia de los mismos. Lo más simple sería permitir que las transacciones se ejecuten secuencialmente, pero existen motivos para permitir la concurrencia:

- **Mejora la utilización de disco y procesador:** aumentando la productividad.
- **Mejora los tiempos de respuesta:** transacciones cortas no necesitan esperar a que se terminen de ejecutar transacciones largas.

# Ejecuciones concurrentes

Cuando se están ejecutando **varias transacciones de manera concurrente**, se puede **perder la consistencia de la base de datos** a pesar de que cada transacción individual sea correcta.

El SGBD debe controlar la interacción entre las transacciones concurrentes para evitar que se destruya la consistencia de la BD mediante **esquemas de control de concurrencia** (mecanismos para lograr **aislamiento**).

Retomemos devuelta el sistema bancario simplificado, en el cual hay un conjunto de transacciones que acceden y modifican las cuentas realizando transferencias de una cuenta a otra.

Consideremos  $T_1$  y  $T_2$ , que son dos transacciones para transferir fondos de una cuenta a otra.

# Ejecuciones concurrentes

$T_1$  transfiere \$500 de la cuenta  $A$  a la cuenta  $B$

$T_1$ :

```
leer(A);  
A := A - 500;  
escribir(A);  
leer(B);  
B := B + 500;  
escribir(B);
```

$T_2$  transfiere el 10% del saldo de la cuenta  $A$  a la cuenta  $B$

$T_2$ :

```
leer(A);  
temp := A * 0.1;  
A := A - temp;  
escribir(A);  
leer(B);  
B := B + temp;  
escribir(B);
```

# Planificación #1: en serie “ $T_1$ ; $T_2$ ”

Supongamos los siguientes valores actuales de las cuentas:  $A = 10.000$  y  $B = 20.000$   
Si las dos transacciones se ejecutan de una en una en el orden  $T_1$  seguida de  $T_2$ , la secuencia sería la siguiente:

| $T_1$                                                                                        |                                                                      | $T_2$                                                                                                                 |                                                                            |
|----------------------------------------------------------------------------------------------|----------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------|
| leer(A);<br>$A := A - 500$ ;<br>escribir(A);<br>leer(B);<br>$B := B + 500$ ;<br>escribir(B); | 10.000<br>9.500<br><b>9.500</b><br>20.000<br>20.500<br><b>20.500</b> |                                                                                                                       |                                                                            |
|                                                                                              |                                                                      | leer(A);<br>$temp := A * 0.1$ ;<br>$A := A - temp$ ;<br>escribir(A);<br>leer(B);<br>$B := B + temp$ ;<br>escribir(B); | 9.500<br>950<br>8.550<br><b>8.550</b><br>20.500<br>21.450<br><b>21.450</b> |

En la secuencia se mantiene la suma  
 $A + B = 30.000$

$9500 + 20500 = 30000$

$8550 + 21450 = 30000$



# Planificación #2: en serie “ T<sub>2</sub>; T<sub>1</sub> ”

Si las dos transacciones se ejecutan en el orden T<sub>2</sub> seguida de T<sub>1</sub>, la secuencia sería la siguiente:

| T <sub>1</sub>                                                                         |                                                                     | T <sub>2</sub>                                                                                               |                                                                               |                                                                                                               |
|----------------------------------------------------------------------------------------|---------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------|
|                                                                                        |                                                                     | leer(A);<br>temp := A * 0.1;<br>A := A - temp;<br>escribir(A);<br>leer(B);<br>B := B + temp;<br>escribir(B); | 10.000<br>1.000<br>9.000<br><b>9.000</b><br>20.000<br>21.000<br><b>21.000</b> |                                                                                                               |
| leer(A);<br>A := A - 500;<br>escribir(A);<br>leer(B);<br>B := B + 500;<br>escribir(B); | 9.000<br>8.500<br><b>8.500</b><br>21.000<br>21.500<br><b>21.500</b> |                                                                                                              |                                                                               | En la secuencia se mantiene la suma<br>A + B = 30.000<br><br>9000 + 21000 = 30000<br><br>8500 + 21500 = 30000 |

# Ejecuciones concurrentes - Planificaciones

Las secuencias de ejecución que vimos se denominan **planificaciones**, y representan el **orden cronológico en el que se ejecutan las instrucciones de transacciones concurrentes en el sistema**.

Las planificaciones anteriores son **secuenciales** porque las instrucciones de cada transacción aparecen todas juntas (de forma consecutiva).

Para un conjunto de  $n$  transacciones, existen  $n!$  planificaciones secuenciales válidas.

# Ejecuciones concurrentes - Planificaciones

Cuando se ejecutan concurrentemente varias transacciones, la planificación no es necesariamente secuencial. Se pueden ejecutar de forma intercalada las instrucciones de las transacciones compartiendo el tiempo de CPU.

- La planificación de un conjunto de transacciones debe considerar **todas las instrucciones** de las mismas.
- Toda planificación **debe respetar el orden en el que se presentan las instrucciones de cada transacción individual**.
- Una planificación concurrente es serializable cuando el resultado de su ejecución equivale a alguna ejecución en serie.

# Planificación #3: concurrente serializable

Supongamos que ahora  $T_1$  y  $T_2$  se ejecutan de forma concurrente de acuerdo a la siguiente planificación

| $T_1$                                        |                                   | $T_2$                                                                |                                       |
|----------------------------------------------|-----------------------------------|----------------------------------------------------------------------|---------------------------------------|
| leer(A);<br>$A := A - 500$ ;<br>escribir(A); | 10.000<br>9.500<br><b>9.500</b>   |                                                                      |                                       |
|                                              |                                   | leer(A);<br>$temp := A * 0.1$ ;<br>$A := A - temp$ ;<br>escribir(A); | 9.500<br>950<br>8.550<br><b>8.550</b> |
| leer(B);<br>$B := B + 500$ ;<br>escribir(B); | 20.000<br>20.500<br><b>20.500</b> |                                                                      |                                       |
|                                              |                                   | leer(B);<br>$B := B + temp$ ;<br>escribir(B);                        | 20.500<br>21.450<br><b>21.450</b>     |

En esta planificación concurrente **se mantiene la consistencia en la BD**, porque el resultado de la ejecución es igual al resultado de la planificación secuencial 1, donde primero ejecutaba  $T_1$  y luego  $T_2$ .

# Planificación #4: concurrente no serializable

Pero no todas las planificaciones concurrentes llevan a un estado consistente de la BD.

| T <sub>1</sub>                                            |                                                   | T <sub>2</sub>                                                                            |                                                    |
|-----------------------------------------------------------|---------------------------------------------------|-------------------------------------------------------------------------------------------|----------------------------------------------------|
| leer(A);<br>A := A - 500;                                 | 10.000<br>9.500                                   |                                                                                           |                                                    |
|                                                           |                                                   | leer(A);<br><i>temp</i> := A * 0.1;<br>A := A - <i>temp</i> ;<br>escribir(A);<br>leer(B); | 10.000<br>1.000<br>9.000<br><b>9.000</b><br>20.000 |
| escribir(A);<br>leer(B);<br>B := B + 500;<br>escribir(B); | <b>9.500</b><br>20.000<br>20.500<br><b>20.500</b> |                                                                                           |                                                    |
|                                                           |                                                   | B := B + <i>temp</i> ;<br>escribir(B);                                                    | 21.000<br><b>21.000</b>                            |

En esta planificación concurrente **no se mantiene la consistencia en la BD**, porque **el resultado de la ejecución no es equivalente al de una planificación secuencial**, ya que generó que A=9.500 y B=21.000, es decir, hay 500 pesos más luego de la ejecución de las transacciones.

# Problemas de la concurrencia

| T <sub>1</sub>                                            |                                                   | T <sub>2</sub>                                                                            |                                                    |
|-----------------------------------------------------------|---------------------------------------------------|-------------------------------------------------------------------------------------------|----------------------------------------------------|
| leer(A);<br>A := A - 500;                                 | 10.000<br>9.500                                   |                                                                                           |                                                    |
|                                                           |                                                   | leer(A);<br><i>temp</i> := A * 0.1;<br>A := A - <i>temp</i> ;<br>escribir(A);<br>leer(B); | 10.000<br>1.000<br>9.000<br><b>9.000</b><br>20.000 |
| escribir(A);<br>leer(B);<br>B := B + 500;<br>escribir(B); | <b>9.500</b><br>20.000<br>20.500<br><b>20.500</b> |                                                                                           |                                                    |
|                                                           |                                                   | B := B + <i>temp</i> ;<br>escribir(B);                                                    | 21.000<br><b>21.000</b>                            |

**Problema de actualización  
perdida**

**Se pierde!!**

# Problemas de la concurrencia

## Problema de actualización temporal.

| T <sub>1</sub>                            |                          | T <sub>2</sub>                                                                |                                |
|-------------------------------------------|--------------------------|-------------------------------------------------------------------------------|--------------------------------|
| leer(A);<br>A := A - 500;<br>escribir(A); | 10.000<br>9.500<br>9.500 |                                                                               |                                |
|                                           | Dato "sucio"!!           | leer(A);<br><i>temp</i> := A * 0.1,<br>A := A - <i>temp</i> ;<br>escribir(A); | 9.500<br>950<br>8.550<br>8.550 |
| leer(B);                                  | 400                      |                                                                               |                                |
|                                           |                          | ...                                                                           |                                |

T<sub>1</sub> actualiza A en la BD y luego falla, pero antes de que se restaure el valor original de A, T<sub>2</sub> leyó el valor temporal de A.

# Problemas de la concurrencia

## Resumen incorrecto

| $T_1$                                     | $T_3$                                                          |
|-------------------------------------------|----------------------------------------------------------------|
|                                           | suma := 0                                                      |
| leer(A);<br>A := A - 500;<br>escribir(A); |                                                                |
|                                           | leer(A);<br>suma := suma + A;<br>leer(B);<br>suma := suma + B; |
| leer(B);<br>B := B + 500;<br>escribir(B); |                                                                |

**T3 lee A después de restarle 500, pero lee B antes de sumarle 500. El resultado es un resumen incorrecto.**



# Problemas de la concurrencia

## Lectura no repetible

| $T_1$                                        | $T_4$                  |
|----------------------------------------------|------------------------|
|                                              | leer(A);               |
| leer(A);<br>$A := A - 500$ ;<br>escribir(A); |                        |
|                                              | ...<br>leer(A);<br>... |
| leer(B);<br>$B := B + 500$ ;<br>escribir(B); |                        |

T4 lee A **antes** de  
restar 500

T4 lee A **después**  
de restar 500

**T4 lee A dos veces, y otra transacción (T1) modifica A entre las dos lecturas:  
T4 recibe diferentes valores para el mismo elemento.**

# Serializabilidad

El SGBD debe controlar la ejecución concurrente de las transacciones para asegurar que el estado de la base sigue siendo consistente. En lo que sigue, asumimos una vista simplificada de las transacciones:

- Desde el punto de vista de la planificación, las instrucciones **leer** y **escribir** son las únicas operaciones significativas de la transacción. Se ignoran las demás operaciones.
- Se asume que entre una instrucción **leer(Q)** y otra **escribir(Q)** sobre un elemento de datos **Q**, una transacción puede realizar una cantidad arbitraria de operaciones con la copia **Q** que reside en la memoria intermedia local de dicha transacción (buffer local).

# Serializabilidad en cuanto a conflictos

Considérese una planificación  $P$  en la cual hay dos **instrucciones consecutivas**  $I_i$  e  $I_j$ , pertenecientes a las transacciones  $T_i$  y  $T_j$  respectivamente.

- Si  $I_i$  e  $I_j$  **se refieren a distintos elementos de datos** se pueden intercambiar  $I_i$  e  $I_j$  sin afectar al resultado de cualquier instrucción de la planificación.
- Si  $I_i$  e  $I_j$  **se refieren al mismo elemento  $Q$**  entonces el orden de los dos pasos puede ser importante. Dado que sólo se tienen en cuenta las instrucciones *leer* y *escribir*:
  - Si **ambas instrucciones** son *leer* entonces **no hay conflicto**
  - El **conflicto** se produce **cuando ambas refieren al mismo dato  $Q$  y al menos una de las instrucciones  $I_i$  o  $I_j$  es *escribir*( $Q$ )**

# Serializabilidad en cuanto a conflictos

Si tenemos dos instrucciones **consecutivas**  $I_i$  e  $I_j$

1.  $I_i = \text{leer}(Q)$ ,  $I_j = \text{leer}(Q)$ : El orden no importa.  $T_i$  y  $T_j$  leen el mismo valor de  $Q$ , independientemente del orden de las lecturas.
2.  $I_i = \text{leer}(Q)$ ,  $I_j = \text{escribir}(Q)$ : Si  $I_i$  está antes que  $I_j$ , entonces  $T_i$  no lee el valor de  $Q$  que escribe la instrucción  $I_j$  de  $T_j$ . Si  $I_j$  está antes que  $I_i$ , entonces  $T_i$  lee el valor de  $Q$  escrito por  $T_j$ . Por tanto, **el orden de  $I_i$  e  $I_j$  es importante**.
3.  $I_i = \text{escribir}(Q)$ ,  $I_j = \text{leer}(Q)$ : Caso análogo al anterior.
4.  $I_i = \text{escribir}(Q)$ ,  $I_j = \text{escribir}(Q)$ : El orden de las instrucciones afecta el valor que obtendrá la siguiente instrucción  $\text{leer}(Q)$  de la planificación  $P$ .

Se dice que  $I_i$  e  $I_j$  están en **conflicto** si existen operaciones de diferentes transacciones sobre el mismo elemento de datos, y al menos una de esas instrucciones es una operación **escribir**.

# Serializabilidad en cuanto a conflictos

Si tenemos dos instrucciones de distintas transacciones  $I_i$  e  $I_j$  que no están en conflicto, podemos cambiar su orden en la planificación y el resultado no se altera:

| Planificación A                              |                                   |                                                                      |                                       |
|----------------------------------------------|-----------------------------------|----------------------------------------------------------------------|---------------------------------------|
| $T_1$                                        |                                   | $T_2$                                                                |                                       |
| leer(A);<br>$A := A - 500$ ;<br>escribir(A); | 10.000<br>9.500<br><b>9.500</b>   |                                                                      |                                       |
|                                              |                                   | leer(A);<br>$temp := A * 0.1$ ;<br>$A := A - temp$ ;<br>escribir(A); | 9.500<br>950<br>8.550<br><b>8.550</b> |
| leer(B);<br>$B := B + 500$ ;<br>escribir(B); | 20.000<br>20.500<br><b>20.500</b> |                                                                      |                                       |
|                                              |                                   | leer(B);<br>$B := B + temp$ ;<br>escribir(B);                        | 20.500<br>21.450<br><b>21.450</b>     |

| Planificación B                              |                                   |                                                               |                                                   |
|----------------------------------------------|-----------------------------------|---------------------------------------------------------------|---------------------------------------------------|
| $T_1$                                        |                                   | $T_2$                                                         |                                                   |
| leer(A);<br>$A := A - 500$ ;<br>escribir(A); | 10.000<br>9.500<br><b>9.500</b>   |                                                               |                                                   |
|                                              |                                   | leer(A);<br>$temp := A * 0.1$ ;<br>$A := A - temp$ ;          | 9.500<br>950<br>8.550                             |
| leer(B);<br>$B := B + 500$ ;<br>escribir(B); | 20.000<br>20.500<br><b>20.500</b> |                                                               |                                                   |
|                                              |                                   | escribir(A);<br>leer(B);<br>$B := B + temp$ ;<br>escribir(B); | <b>8.550</b><br>20.500<br>21.450<br><b>21.450</b> |

# Serializabilidad en cuanto a conflictos

Si una planificación  $P$  se puede transformar en otra  $P'$  por medio de una serie de intercambios de instrucciones no conflictivas, se dice que  $P$  y  $P'$  son **equivalentes en cuanto a conflictos**.

Se dice que una planificación  $P$  es **serializable en cuanto a conflictos** si es **equivalente en cuanto a conflictos** a una planificación secuencial.

EJ:

| T3          | T4          |
|-------------|-------------|
| leer(Q)     |             |
|             | escribir(Q) |
| escribir(Q) |             |

Esta planificación sólo consta de las operaciones significativas de las transacciones T3 y T4 (es decir, leer y escribir). Esta planificación **no es secuenciable en cuanto a conflictos**, ya que no es equivalente ni a la planificación secuencial  $\langle T3, T4 \rangle$  ni a  $\langle T4, T3 \rangle$ .

# Pruebas de serializabilidad

- Una planificación es serializable en conflictos si se puede transformar intercambiando instrucciones consecutivas en una planificación en serie, o lo que es igual, si el resultado obtenido de la planificación es equivalente al resultado obtenido de una planificación en serie.
- Sin embargo, no es eficiente intentar alcanzar una planificación en serie “probando” este tipo de intercambios.
- Existe un algoritmo simple y eficiente que determina si una planificación  $P$  es serializable: **construir un grafo de precedencia** para  $P$  y evalúa las propiedades del grafo.

# Grafo de precedencia

El grafo de precedencia consiste en un par  $\mathbf{G} = (\mathbf{V}, \mathbf{A})$ . Donde  $\mathbf{V}$  es un conjunto de vértices y  $\mathbf{A}$  un conjunto de arcos (o aristas dirigidas).

- El conjunto de **vértices** consiste en todas las **transacciones que participan en la planificación**. Debemos crear un nodo por cada  $T$  de la planificación.
- El conjunto de **arcos** consiste en todos los arcos  $T_i$  a  $T_j$  ( $T_i \rightarrow T_j$ ) **por precedencia de operaciones leer(Q)/escribir(Q)**

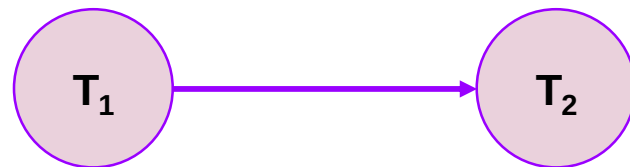
Se añade un arco  $T_i \rightarrow T_j$  cuando se da alguna de las siguientes condiciones:

- $T_i$  ejecuta *escribir(Q)* antes que  $T_j$  ejecute *leer(Q)*
- $T_i$  ejecuta *leer(Q)* antes que  $T_j$  ejecute *escribir(Q)*
- $T_i$  ejecuta *escribir(Q)* antes que  $T_j$  ejecute *escribir(Q)*



# Planificación #1: Grafo de precedencia

| $T_1$                                                                                  |                                                                      | $T_2$                                                                                                                               |                                                                            |
|----------------------------------------------------------------------------------------|----------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------|
| leer(A);<br>A := A - 500;<br>escribir(A);<br>leer(B);<br>B := B + 500;<br>escribir(B); | 10.000<br>9.500<br><b>9.500</b><br>20.000<br>20.500<br><b>20.500</b> |                                                                                                                                     |                                                                            |
|                                                                                        |                                                                      | leer(A);<br><i>temp</i> := A * 0.1;<br>A := A - <i>temp</i> ;<br>escribir(A);<br>leer(B);<br>B := B + <i>temp</i> ;<br>escribir(B); | 9.500<br>950<br>8.550<br><b>8.550</b><br>20.500<br>21.450<br><b>21.450</b> |



El arco  $T_1 \rightarrow T_2$  existe porque las escrituras de  $T_1$  preceden a las lecturas y escrituras de  $T_2$

# Planificación #2: Grafo de precedencia

| $T_1$                                                                                  |                                                                     | $T_2$                                                                                                                               |                                                                               |
|----------------------------------------------------------------------------------------|---------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------|
|                                                                                        |                                                                     | leer(A);<br><i>temp</i> := A * 0.1;<br>A := A - <i>temp</i> ;<br>escribir(A);<br>leer(B);<br>B := B + <i>temp</i> ;<br>escribir(B); | 10.000<br>1.000<br>9.000<br><b>9.000</b><br>20.000<br>21.000<br><b>21.000</b> |
| leer(A);<br>A := A - 500;<br>escribir(A);<br>leer(B);<br>B := B + 500;<br>escribir(B); | 9.000<br>8.500<br><b>8.500</b><br>21.000<br>21.500<br><b>21.500</b> |                                                                                                                                     |                                                                               |



El arco  $T_2 \rightarrow T_1$  existe porque las escrituras de  $T_2$  preceden a las lecturas y escrituras de  $T_1$

# Planificación #3: Grafo de precedencia

| T <sub>1</sub>                            |                                   | T <sub>2</sub>                                                 |                                       |
|-------------------------------------------|-----------------------------------|----------------------------------------------------------------|---------------------------------------|
| leer(A);<br>A := A - 500;<br>escribir(A); | 10.000<br>9.500<br><b>9.500</b>   |                                                                |                                       |
|                                           |                                   | leer(A);<br>temp := A * 0.1;<br>A := A - temp;<br>escribir(A); | 9.500<br>950<br>8.550<br><b>8.550</b> |
| leer(B);<br>B := B + 500;<br>escribir(B); | 20.000<br>20.500<br><b>20.500</b> |                                                                |                                       |
|                                           |                                   | leer(B);<br>B := B + temp;<br>escribir(B);                     | 20.500<br>21.450<br><b>21.450</b>     |



Las dos lecturas de T<sub>2</sub>  
deben ser posteriores a  
las dos escrituras de T<sub>1</sub>

# Planificación #4: Grafo de precedencia

| $T_1$                                                                  |                                                   | $T_2$                                                                                                |                                                    |
|------------------------------------------------------------------------|---------------------------------------------------|------------------------------------------------------------------------------------------------------|----------------------------------------------------|
| leer(A); <b>(1)</b><br>$A := A - 500;$                                 | 10.000<br>9.500                                   |                                                                                                      |                                                    |
|                                                                        |                                                   | leer(A);<br>$temp := A * 0.1;$<br>$A := A - temp;$<br>escribir(A); <b>(2)</b><br>leer(B); <b>(3)</b> | 10.000<br>1.000<br>9.000<br><b>9.000</b><br>20.000 |
| escribir(A);<br>leer(B);<br>$B := B + 500;$<br>escribir(B); <b>(4)</b> | <b>9.500</b><br>20.000<br>20.500<br><b>20.500</b> |                                                                                                      |                                                    |
|                                                                        |                                                   | $B := B + temp;$<br>escribir(B);                                                                     | 21.000<br><b>21.000</b>                            |



El arco de  $T_1 \rightarrow T_2$  existe porque **(1)** debe preceder a **(2)**, y el arco de  $T_2 \rightarrow T_1$  existe porque **(3)** debe preceder a **(4)**.

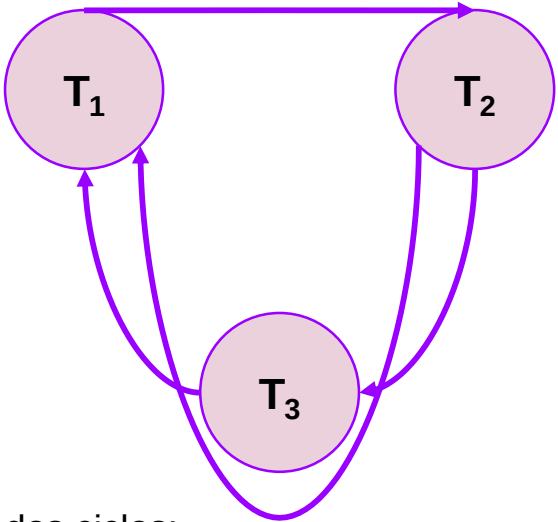
# Detección de la serializabilidad por conflictos

- Un arco  $T_i \rightarrow T_j$  indica que  $T_i$  debe aparecer **antes** que  $T_j$  en una **planificación serie equivalente** a  $P$ , porque hay dos operaciones en conflicto que aparecen en dicho orden.
- Si el grafo contiene un **ciclo**,  **$P$  no es serializable por conflictos**.
  - Un ciclo es una secuencia de arcos de la forma:
$$T_i \rightarrow T_j, T_j \rightarrow T_k, \dots, T_p \rightarrow T_i$$
- Si **no hay ciclos** en el grafo, entonces  **$P$  es serializable**

# Ejemplo de planificación no serializable

| Transacción T <sub>1</sub>                          | Transacción T <sub>2</sub>                                       | Transacción T <sub>3</sub>                           |
|-----------------------------------------------------|------------------------------------------------------------------|------------------------------------------------------|
| leer(A);<br>escribir(A);<br>leer(B);<br>escribir(B) | leer(C);<br>leer(B);<br>escribir(B);<br>leer(A);<br>escribir(A); | leer(B);<br>leer(C);<br>escribir(B);<br>escribir(C); |

| T <sub>1</sub>           | T <sub>2</sub>                       | T <sub>3</sub>               |
|--------------------------|--------------------------------------|------------------------------|
|                          | leer(C);<br>leer(B);<br>escribir(B); |                              |
|                          |                                      | leer(B);<br>leer(C);         |
| leer(A);<br>escribir(A); |                                      |                              |
|                          |                                      | escribir(B);<br>escribir(C); |
|                          | leer(A);                             |                              |
| leer(B);<br>escribir(B); |                                      |                              |



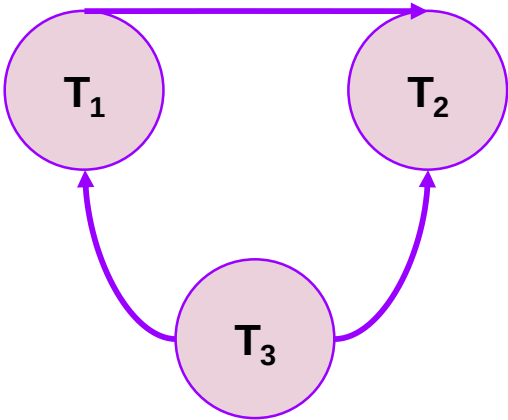
Hay dos ciclos:

$$T_1 \rightarrow T_2 \rightarrow T_1$$

$$T_1 \rightarrow T_2 \rightarrow T_3 \rightarrow T_1$$

# Ejemplo de planificación serializable

| Transacción T <sub>1</sub>                          | Transacción T <sub>2</sub>                                       | Transacción T <sub>3</sub>                           |
|-----------------------------------------------------|------------------------------------------------------------------|------------------------------------------------------|
| leer(A);<br>escribir(A);<br>leer(B);<br>escribir(B) | leer(C);<br>leer(B);<br>escribir(B);<br>leer(A);<br>escribir(A); | leer(B);<br>leer(C);<br>escribir(B);<br>escribir(C); |



La planificación serie  
equivalente es:

$$T_3 \rightarrow T_1 \rightarrow T_2$$

| T <sub>1</sub>           | T <sub>2</sub>                       | T <sub>3</sub>               |
|--------------------------|--------------------------------------|------------------------------|
|                          |                                      | leer(B);<br>leer(C);         |
| leer(A);<br>escribir(A); |                                      |                              |
|                          |                                      | escribir(B);<br>escribir(C); |
|                          | leer(C);                             |                              |
| leer(B);<br>escribir(B); |                                      |                              |
|                          | leer(B);<br>escribir(B);<br>leer(A); |                              |

# ¿Cómo se supervisa la concurrencia en el

SO

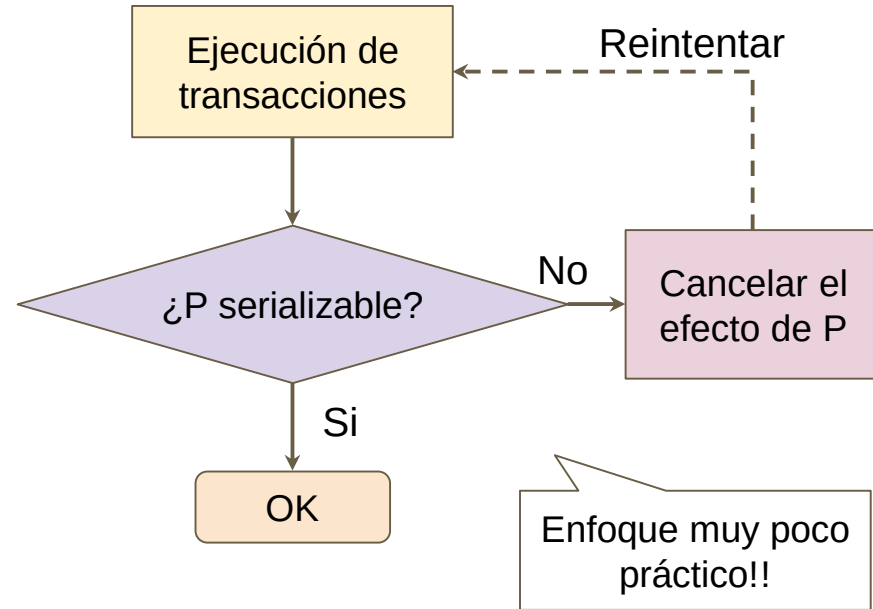
Planificador de  
Tareas del SO

Planificación P  
(ordenamiento de  
las operaciones)

- **Carga** del sistema
- Momento de introducción de las transacciones
- **Prioridades** de los procesos
- ...

Es el SO quien distribuye los recursos para los procesos, y determina la intercalación de las operaciones de las transacciones concurrentes (ejecutadas como procesos del SO)

Entonces, habría que comprobar si P es serializable una vez que se ejecutan las transacciones incluidas en P...



Esto es **ineficiente**, debemos encontrar **técnicas** que **garanticen la seriabilidad** sin tener que verificar a posteriori.



# Técnicas de control de concurrencia

Son **métodos basados en la teoría de la serializabilidad**, que definen un conjunto de **reglas (protocolos)** tal que...

- Si todas las transacciones las cumplen, o
- Si el subsistema de control de concurrencia del SGBD las impone automáticamente

**... se asegura la serializabilidad de toda planificación** de transacciones.

Se clasifican en:

- Métodos de **bloqueo**
- Métodos de **marca de tiempo**
- Técnicas de **multiversión**
- Métodos **optimistas**

# Métodos de bloqueo

Utiliza **bloqueos** para controlar el acceso concurrente a los elementos de datos almacenados en la base de datos

Reglas básicas del bloqueo:

- **Bloqueo compartido / para lectura**: si una transacción tiene un bloqueo compartido sobre un elemento de datos, **puede leer el elemento pero no actualizarlo** (escribir).
  - Varias transacciones pueden mantener a la vez bloqueos compartidos sobre el mismo elemento
- **Bloqueo exclusivo / para escritura**: si una transacción tiene un bloqueo exclusivo sobre un elemento de datos, **puede leer y actualizar** (escribir) **el elemento**.
  - Un bloqueo exclusivo proporciona acceso exclusivo al elemento

# Métodos de bloqueo - Reglas

1. **T** debe emitir **bloquear\_lectura(X)** o **bloquear\_escritura(X)** antes de ejecutar una operación **leer(X)**
2. **T** debe emitir **bloquear\_escritura(X)** antes de realizar una operación **escribir(X)** en **T**.
3. **T** debe emitir **desbloquear(X)** una vez completadas todas las operaciones **leer(X)** y **escribir(X)**
4. Si **T** ya posee un bloqueo, compartido o exclusivo, sobre X no emitirá **bloquear\_lectura(X)** ni **bloquear\_escritura(X)**.
  - a. \*esta regla puede permitir excepciones: mejora y reducción de bloqueos.
5. **T** no emitirá **desbloquear(X)** a menos que posea un bloqueo, compartido o exclusivo, sobre X.

# Métodos de bloqueo

- Cuando una transacción T solicita un bloqueo...
  - Se le concede el bloqueo si el elemento no ha sido ya bloqueado por otra transacción.
  - Si el elemento está bloqueado por otra transacción, el SGBD determina si la solicitud es compatible con el bloqueo existente:
    - Si se pide un bloqueo compartido sobre un elemento que ya tiene un bloqueo compartido, el bloqueo será concedido a T
    - En cualquier otro caso, T debe esperar hasta que se libere el bloqueo existente.
- Una transacción que obtiene un bloqueo lo mantiene hasta que lo libera explícitamente o termina (*commit* o *rollback*)
  - Sólo cuando se libera un bloqueo exclusivo los efectos de la escritura serán visibles

# Métodos de bloqueo

- Algunos sistemas permiten la **mejora** (o promoción) y la **reducción** (o degradación) de bloqueos.
  - Esto aumenta el nivel de concurrencia del sistema.
- Si T emitió bloquear\_lectura(X), más tarde puede mejorarlo a bloqueo exclusivo emitiendo bloquear\_escritura(X)
  - Si T es la única que tiene un bloqueo compartido sobre X se le concede la solicitud
  - En otro caso, T debe esperar.
- Si T emitió bloquear\_escritura(X), más tarde puede reducirlo a bloqueo compartido emitiendo bloquear\_lectura(X).
  - De esta forma permite que otras transacciones lean X.

# Métodos de bloqueo

El uso de bloqueos para la programación de transacciones no garantiza la serializabilidad de las planificaciones:

| Transacción T <sub>4</sub>                                                                                                                                              | Transacción T <sub>5</sub>                                                                                                                                              |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>bloquear_lectura(Y);</b><br>leer(Y);<br><b>desbloquear(Y);</b><br><b>bloquear_escritura(X);</b><br>leer(X);<br>X := X + Y;<br>escribir(X);<br><b>desbloquear(X);</b> | <b>bloquear_lectura(X);</b><br>leer(X);<br><b>desbloquear(X);</b><br><b>bloquear_escritura(Y);</b><br>leer(Y);<br>Y := X + Y;<br>escribir(Y);<br><b>desbloquear(Y);</b> |

Valores iniciales: X=20, Y=30

Resultados de las **planificaciones serie**:

T<sub>4</sub> → T<sub>5</sub> : X = 50; Y = 80

T<sub>5</sub> → T<sub>4</sub> : X = 70; Y = 50

| Planificación P                                                                      |                                                                                                                                             |
|--------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------|
| T <sub>4</sub>                                                                       | T <sub>5</sub>                                                                                                                              |
| bloquear_lectura(Y);<br>leer(Y);<br>desbloquear(Y);                                  |                                                                                                                                             |
|                                                                                      | bloquear_lectura(X);<br>leer(X);<br>desbloquear(X);<br>bloquear_escritura(Y);<br>leer(Y);<br>Y := X + Y;<br>escribir(Y);<br>desbloquear(Y); |
| bloquear_escritura(X);<br>leer(X);<br>X := X + Y;<br>escribir(X);<br>desbloquear(X); |                                                                                                                                             |

Resultados de la **planificacion P**: **X=50; Y=50. (No serializable!)**

# Métodos de bloqueo: Bloqueo en dos fases

- Es necesario seguir un **protocolo adicional** que indique **dónde colocar las operaciones de bloqueo y desbloqueo** dentro de las transacciones.
- El más conocido es el **Bloqueo en Dos Fases** (B2F)
- Una transacción T sigue el protocolo de bloqueo en dos fases si **todas las operaciones de bloqueo preceden a la primera operación de desbloqueo**.
- De este modo podemos ver a T dividida en dos fases:
  - **Fase de expansión** (o crecimiento)
    - T puede adquirir bloqueos
    - T no puede liberar ningún bloqueo
  - **Fase de contracción**
    - T puede liberar bloqueos existentes

# Métodos de bloqueo: Bloqueo en dos fases

- Si el sistema permite mejorar y reducir bloqueos:
  - La mejora sólo puede tener lugar en la fase de expansión
  - La reducción sólo puede realizarse en la fase de contracción
- En el código de T, un `bloquear_lectura(X)` puede aparecer en la fase de contracción de T sólo si reduce un bloqueo exclusivo a uno compartido.

| Transacción T <sub>4</sub>                                                                                                                                                                                                                          | Transacción T <sub>5</sub>                                                                                                                                                                                                                          |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>bloquear_lectura(Y);</code><br><code>leer(Y);</code><br><code>bloquear_escritura(X);</code><br><code>desbloquear(Y);</code><br><code>leer(X);</code><br><code>X := X + Y;</code><br><code>escribir(X);</code><br><code>desbloquear(X);</code> | <code>bloquear_lectura(X);</code><br><code>leer(X);</code><br><code>bloquear_escritura(Y);</code><br><code>desbloquear(X);</code><br><code>leer(Y);</code><br><code>Y := X + Y;</code><br><code>escribir(Y);</code><br><code>desbloquear(Y);</code> |



# Métodos de bloqueo: Bloqueo en dos fases

- Si toda transacción de una planificación sigue el protocolo de bloqueo en dos fases, entonces la planificación es serializable.
- Ventaja: ya no es necesario comprobar la seriabilidad de las planificaciones!
- Desventaja:
  - El B2F puede limitar el grado de concurrencia en un plan
  - Emplear bloqueos puede provocar problemas de:
    - **Interbloqueo** (bloqueo mortal - deadlock)
    - **Bloqueo indefinido** (o espera indefinida)

# Bloqueo en dos fases: variantes

- Bloqueo en dos fases **Conservador o Estático**

- T debe **declarar y bloquear todos los elementos** a los que tendrá acceso (lectura o escritura) **antes de comenzar** a ejecutarse.
  - Si no es posible bloquear algún elemento, T no bloqueará ninguno y esperará para reintentarlo más tarde.
  - Protocolo libre de interbloqueo

- Bloqueo en dos fases **Estricto** (el más utilizado)

- T **no libera ningún bloqueo exclusivo hasta terminar** (con COMMIT o ROLLBACK)
  - Ninguna transacción lee o escribe un elemento **modificado** por T hasta que T se haya completado
  - Puede sufrir interbloqueo (salvo que se combine con B2F conservador)

- Bloqueo en dos fases **Riguroso** (más restrictivo que B2F estricto)

- T **no libera ningún bloqueo (compartido ni exclusivo) hasta terminar** (con COMMIT o ROLLBACK)

# El problema del interbloqueo

Es la situación en la que cada una de dos (o más) transacciones está esperando a que se libere un bloqueo establecido por otra transacción

El SGBD debe detectar interbloqueos y romperlos:

- Abortar una o más transacciones: se deshacen sus escrituras y se liberan sus bloqueos, así el resto de las transacciones puede continuar su ejecución
- Reiniciar automáticamente las transacciones abortadas.

| Planificación                                                     |                                                                                                                              |
|-------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------|
| T <sub>6</sub>                                                    | T <sub>7</sub>                                                                                                               |
| bloquear_escritura(Y);<br>leer(Y);<br>Y := Y - 10;<br>escribir(Y) |                                                                                                                              |
|                                                                   | bloquear_escritura(X);<br>leer(X);<br>X := X + 100;<br>escribir(X)<br><b>bloquear_escritura(Y);</b><br><b>[en espera...]</b> |
| <b>bloquear_escritura(X);</b><br><b>[en espera...]</b>            |                                                                                                                              |



# El problema del interbloqueo

Hay 3 técnicas generales para gestionar los interbloqueos:

- **Temporizaciones** de bloqueos
- **Prevención** de interbloqueos (más difícil y costoso, no se utiliza en la práctica)
- **Detección** de interbloqueos

Conviene **detectar** interbloqueos cuando se sabe que hay poca interferencia entre transacciones, es decir si:

- las transacciones son cortas y bloquean pocos elementos, o
- la carga de transacciones es pequeña.

En otro caso, conviene usar **temporizaciones**.

# Temporizaciones de bloqueos

Una transacción que solicita un bloqueo sólo esperará durante un período de tiempo predefinido por el sistema.

Si no se le concede el bloqueo durante ese tiempo, se producirá un “fin de temporización”: el SGBD asumirá que la transacción está interbloqueada (aunque puede que no), la abortará y la reiniciará automáticamente.

Es una solución muy sencilla y práctica, pero puede hacer que sean abortadas y reiniciadas transacciones que en realidad no están en un interbloqueo.

# Detección de interbloqueos

Verificación periódica del estado del sistema: ¿Está en un bloqueo mortal?

Creación de un **grafo de espera** que muestra las dependencias entre las transacciones:

- Crear un **nodo** por cada **transacción** en ejecución, etiquetado con el identificador de la transacción  $T$   

- Si  $T_j$  espera para bloquear el elemento  $X$ , ya bloqueado por  $T_k$ , crear un **arco** desde  $T_j$  a  $T_k$   

- Cuando  $T_k$  libera el bloqueo sobre  $X$ , **borrar** el arco correspondiente.

Si existe un **ciclo** en el grafo de espera entonces se ha detectado un **interbloqueo** entre las transacciones.

# Detección de interbloqueos

Pero, ¿cuándo hay que verificar el estado del sistema? (ejecutar el algoritmo que genera el grafo de espera)

- A **intervalos uniformes** de tiempo, o
- A **intervalos** de tiempo **desiguales**:
  - Iniciar el algoritmo de detección con un tamaño de intervalo inicial
  - Cada vez que no se detecta un interbloqueo, incrementar el intervalo
    - Por ejemplo, al doble que el anterior
  - Cada vez que se detecta interbloqueo, reducir el intervalo
    - Por ejemplo, a la mitad
  - Existirán límites superior e inferior del tamaño del intervalo.



# Detección de interbloqueos

- Si el sistema está en un estado de interbloqueo, el SGBD necesita abortar algunas transacciones...
- ¿Cuales? → Selección de víctimas
  - Es mejor abortar transacciones que lleven poco tiempo en ejecución
  - Es mejor abortar una transacción que haya hecho pocos cambios en la BD
  - Es mejor abortar una transacción que todavía debe hacer muchos cambios en la BD (puede que el SGBD no conozca esta información)
- Se trata de abortar las transacciones que supongan el mínimo coste.
- Es necesario evitar la **inanición**.

# Detección de interbloqueos: el problema de la inanición

- Una transacción sufre **inanición** cuando es seleccionada para ser abortada (**víctima**) sucesivamente: **nunca termina su ejecución**.
  - Es similar al bloqueo indefinido
- La solución es asignar prioridades más altas a las transacciones abortadas varias veces, para que no sean siempre las víctimas.

# El problema del bloqueo indefinido

- Ocurre cuando el protocolo de control de concurrencia **nunca selecciona a una transacción que está esperando para establecer un bloqueo**, mientras otras transacciones continúan ejecutándose con normalidad.
  - Ocurre si el **esquema de espera** da más prioridad a unas transacciones que a otras
- Dos algoritmos de prevención de bloqueo indefinido
  - Consiguen un **esquema de espera justo**:
  - **El primero en llegar es el primero en ser atendido**: las transacciones pueden bloquear el elemento X en el orden que solicitaron su bloqueo
  - **Aumento de prioridad en la espera**
    - Cuanto más espera T, mayor es su prioridad.
    - Cuando T tiene la prioridad más alta de todas, obtiene el bloqueo y continúa su ejecución.

# Transacciones en PostgreSQL -

## Bloqueos

- La sentencia **BEGIN**; comienza una transacción.
- La sentencia **COMMIT**; termina la transacción y almacena todos los cambios.
- Las sentencias SQL ejecutadas entre BEGIN y COMMIT se ejecutan atómicamente.
- La sentencia **ROLLBACK**; puede utilizarse para retroceder la transacción y volver al estado previo al comienzo de la misma.
- Dentro de una transacción, las sentencias INSERT, DELETE, UPDATE bloquean automáticamente las filas involucradas en modo exclusivo.
  - un detalle de Postgres: Si dentro de un bloque BEGIN ... COMMIT una de las sentencias falla (ej. error de sintaxis o violación de constraint), toda la transacción entra en estado "ABORTED". Postgres rechazará cualquier comando siguiente (incluso los correctos) hasta que hagas un ROLLBACK. Es más estricto que MySQL en este sentido.
- Para la sentencia SELECT debemos indicar el modo en el que vamos a acceder a los datos.

# Transacciones en PostgreSQL - Bloqueos: Sintaxis y USO

Postgres tiene un control más fino sobre los bloqueos en los SELECT:

- **FOR UPDATE**: bloquea las filas involucradas en la consulta en modo exclusivo. Se usa cuando vas a leer un dato con la intención de modificarlo inmediatamente.
- **FOR NO KEY UPDATE** (Exclusivo de Postgres): Bloquea la fila para escritura, pero no bloquea a quienes quieran crear claves foráneas (Foreign Keys) contra esa fila. Es más "amable" que el FOR UPDATE tradicional.
- **FOR SHARE**: Bloqueo compartido (para leer asegurando integridad). Bloquea las filas involucradas en la consulta en modo compartido. Asegura que la fila no sea modificada ni borrada por nadie más mientras la leemos, pero permite que otros también la lean (y hagan FOR SHARE también).
- **FOR KEY SHARE**: Permite aún más concurrencia que FOR SHARE. Es más permisivo, permite cambios en cualquier campo de la fila excepto en la clave primaria.

# Transacciones en PostgreSQL - Ejemplo

-- 1. Iniciamos 'T'

**BEGIN;**

-- 2. Leemos y bloqueamos la fila de la cuenta 10.

-- Nadie podrá modificarla (UPDATE/DELETE) hasta que termine 'T'.

**SELECT \* FROM** cuentas **WHERE** id = 10 **FOR UPDATE;**

-- 3. Hacemos la operación

**UPDATE** cuentas **SET** saldo = saldo - 100 **WHERE** id = 10;

-- 4. Confirmamos (acá se liberan los bloqueos)

**COMMIT;**

# Transacciones en PostgreSQL - Ejemplo

**2**  
-- 1. Iniciamos

**BEGIN;**

-- 2. Leemos y bloqueamos la fila del producto 50.

-- Nadie podrá comprarlo (UPDATE/DELETE) hasta que yo termine.

-- Observación: si usamos **SELECT ... FOR NO KEY UPDATE**, otros sí podrían ver el valor viejo con un **SELECT** normal.

**SELECT \* FROM productos WHERE id = 50 FOR UPDATE;**

-- 3. Hacemos la operación necesaria

**INSERT INTO ventas (producto\_id, cantidad) VALUES (50, 1);**

**UPDATE productos SET stock = stock - 1 WHERE id = 50;**

-- 4. Confirmamos (se liberan los bloqueos)

# Limitaciones de los métodos de bloqueo

## (B2F)

Aunque el Bloqueo en Dos Fases (B2F) garantiza la serializabilidad, tiene un gran problema de rendimiento en entornos de alta concurrencia.

El **cuello de botella** de los bloqueos:

- Los lectores bloquean a los escritores (Un bloqueo compartido impide uno exclusivo).
- Los escritores bloquean a los lectores (Un bloqueo exclusivo impide uno compartido).

En un sistema real (ej. un banco o e-commerce), si una transacción larga está leyendo un reporte, nadie puede actualizar esos datos hasta que termine.

Si un usuario está actualizando un dato, nadie puede leerlo y el sistema se queda "pensando". ¿Cómo evitamos esto?



# Control de Concurrency Multiversión (MVCC)

Para solucionar los problemas del B2F, los SGBD modernos (como PostgreSQL y Oracle) implementan el MVCC (Multi-Version Concurrency Control).

La regla de oro del MVCC:

- Los lectores NO bloquean a los escritores.
- Los escritores NO bloquean a los lectores.

¿Cómo lo logra?

- En lugar de sobrescribir el dato original y bloquearlo, el SGBD crea una nueva versión del elemento de datos cada vez que se actualiza.
- Mantiene múltiples versiones del mismo registro coexistiendo en la base de datos simultáneamente.

# ¿Cómo funciona MVCC? (Snapshots)

- Cuando una transacción inicia, el SGBD le asigna un número de secuencia (Timestamp/Transaction ID) y le toma una "foto" (Snapshot) al estado de la base de datos.
- **Lecturas:** La transacción solo lee las versiones de los datos que ya estaban confirmadas (COMMIT) en el momento en que se tomó su foto.
- **Escrituras:** Si la transacción T1 modifica un dato, no borra el viejo. Crea una versión nueva invisible para los demás.
- Si otra transacción T2 intenta leer ese mismo dato, no tiene que esperar. Simplemente lee la versión "vieja" que le corresponde según su propia foto.

# Enfoque Pesimista vs. Enfoque Optimista

B2F (Pesimista):

- "Asumo que habrá conflictos, por lo tanto, bloqueo el recurso antes de usarlo por si acaso".
- Ventaja: Evita que las transacciones fallen.
- Desventaja: Lentitud y alto riesgo de interbloqueos (Deadlocks).

MVCC (Optimista):

- "Asumo que rara vez chocaremos. Todos trabajan con sus propias versiones. Si al momento de hacer COMMIT resulta que dos transacciones modificaron exactamente el mismo dato, aborto una de las dos".
- Ventaja: Rendimiento altísimo, lecturas instantáneas sin bloqueos.
- Desventaja: El programador debe estar preparado para que el SGBD aborte su transacción (Estado ABORTED) y deba reintentarla.

# Niveles de Aislamiento (SQL Standard)

El estándar SQL define 4 niveles de aislamiento que los SGBD implementan (generalmente usando MVCC) para decidir qué "versión" de los datos ve una transacción:

1. **READ UNCOMMITTED:** Permite "Lecturas Sucias" (leer datos de otras transacciones que aún no hicieron COMMIT). Postgres no lo permite por seguridad.
2. **READ COMMITTED (Por defecto):** Una consulta solo ve datos confirmados. Pero si haces dos SELECT iguales, el dato podría cambiar en el medio (Lectura no repetible).
3. **REPEATABLE READ:** Congela la "foto" al inicio de la transacción. Todos los SELECT verán exactamente los mismos datos siempre, ignorando los COMMITs de los demás.
4. **SERIALIZABLE:** El aislamiento absoluto. Garantiza matemáticamente el mismo resultado que una ejecución en serie. Ante la mínima sospecha de inconsistencia, aborta la transacción.

# MVCC y Bloqueos en PostgreSQL

En el núcleo de PostgreSQL, el control de concurrencia se maneja 100% con MVCC.

Por defecto, las consultas normales (SELECT, UPDATE) usan MVCC en nivel Read Committed para asegurar velocidad masiva sin bloqueos de lectura.

Sin embargo, Postgres permite combinar MVCC con B2F:

- Cuando la lógica de negocio exige control estricto (ej. descontar stock o cobrar saldo), el programador puede forzar bloqueos explícitos a nivel de fila.
- Para esto utilizamos las sentencias `SELECT ... FOR UPDATE` y `SELECT ... FOR SHARE`.

Cuando te dicen:  
"El excel también es una  
base de datos"

